MedEx: Senior Frontend Engineer Take-Home Exercise

Task 2: Multi-provider Cart and Split Checkout

Thank you for your interest in joining MedEx.

MedEx is a healthcare marketplace. Customers in Thailand, Nepal and India use it to find clinics, hospitals and labs, compare health packages, lab tests and wellness treatments, and book them online. On a single visit, one customer often deals with several providers and several currencies.

This exercise is about UI and UX when the data is messy. The data we give you has the same shape as our production API, rough edges included. How you handle those edges, and how you explain your decisions, matters much more to us than how many features you finish.

How this works

- **Time:** spend about 6–8 hours. You have 5 days from the day you receive this. If you run out of time, stop and write in your README what you would do next. That counts in your favour.
- **Stack:** React with TypeScript is required. Next.js is preferred, since it's what we use, but not required. Use any component library, styling approach or state tool you like, and tell us why you chose it.
- **Data:** the task's JSON file is in the `data/` folder next to this brief. All of it is fictional. Don't import the JSON straight into components. Put it behind your own mock API layer, for example MSW, Next.js API routes, or an in-memory service. That layer should have realistic latency, so your UI works the way it would against a real server. The field names and formats match our real API, so expect to map them into the shapes your UI needs.
- **AI tools** are allowed. If you use them, say where in your README. You must be able to explain every line in the follow-up interview.

What to send us

1. A link to a Git repository (GitHub or GitLab) with a commit history we can follow.
2. A README.md covering:
3. how to run it (ideally with one command),
4. the decisions you made and the trade-offs you accepted,
5. anything you deliberately left out, and what you would do with more time,
6. where you used AI tools, if you did.
7. Tests for the logic you consider most important. We don't expect full coverage.
8. Optional but welcome: a deployed link, a 3–5 minute screen recording walking through it, or both.

How we evaluate

Area	What we look for
UX thinking	Every state makes sense: loading, empty, error, partial, slow and offline. A non-technical customer understands what is happening.
Correctness	The logic is right in the awkward cases in the data, not just the happy path.
Architecture	State lives where it belongs, and a teammate could extend your code without fear.
Accessibility	Keyboard use, screen readers, focus management and contrast.
Performance	It stays smooth with realistic data volume.
Communication	Your README is honest and clear about trade-offs.

In the follow-up interview, you'll walk us through your solution and then extend it live for about 30 minutes.

Domain notes

Service types. Every service has a `type`: `package`, `lab_test`, `wellness`, `service` or `insurance`.

Currency belongs to the provider. A service is priced in its provider's currency (THB, NPR or INR). Our system has no exchange rates. Never add amounts in different currencies together, and never convert silently.

Wellness services have no single price. Their price depends on a duration option the customer picks, such as 60 or 90 minutes. For these services `price` is `null`, and the options are listed in `wellnessPricing`. In the cart, each option's `amount_after_service_charge` is already the final price per unit for that duration. A wellness service's price is unknown until an option is chosen.

Providers enter this data by hand. It contains mistakes and inconsistencies. When you find one, decide what the customer should see, handle it, and write the decision down in your README.

Your task

Data file: `data/task2-cart.json`

The story

> "*I put a lab test from one clinic, a facial from a spa, and a check-up package from a hospital in my cart. Then I tried to pay.*"

A MedEx cart can hold services from different providers, of different types, in different currencies. There is no single checkout: each group in the cart is paid for and booked separately, through its own flow. Build the cart and the checkout experience.

Data

The file contains three keys:

- `catalog`: products the customer can add.
- `serverCart`: the logged-in customer's cart, in the shape of our `GET /v2/cart`.
- `guestCart`: what a guest stored on their device before logging in.

Excerpt:

```
// serverCart
{
  "groups": [
    {
      "type": "lab_test",
      "itemCount": 4,               // sum of quantities, not number of lines
      "unpricedItems": 0,
      "subtotals": { "THB": 1677, "NPR": 12520 },
      "items": [
        { "id": "…", "productId": "…", "quantity": 2,
          "product": { "id": "…", "title": "…", "type": "lab_test", "price": 4760,
                       "currency": "NPR", "providerId": 137, "providerName": "…" } },
        …
      ]
    },
    {
      "type": "wellness", "itemCount": 1, "unpricedItems": 1, "subtotals": {},
      "items": [
        { "id": "…", "productId": "5f0c1a52-…-000000000003", "quantity": 1,
          "product": { "title": "Signature Facial", "type": "wellness", "price": null, "priceUnknown":
true,
                       "currency": "THB", "providerId": 61, "providerName": "Lotus Day Spa",
                       "wellnessPricing": [
                         { "id": "…", "durationInMinutes": 60, "amount_after_service_charge": 1800 },
                         { "id": "…", "durationInMinutes": 90, "amount_after_service_charge":
2040 } ] } }
      ]
    },
    { "type": "unknown", "itemCount": 1, "unpricedItems": 1, "subtotals": {},
      "items": [ { "id": "…", "productId": "…", "quantity": 1, "product": {} } ] }
  ],
  "totalItems": 8
}
```

Notes on the data:

- In the cart, `product.price` is already the final price per unit.
- A line whose `product` is empty means the provider has since removed the service.

The mock API you build

Serve the data through these endpoints:

```
GET    /cart                                    -> 200 { cart }
PATCH  /cart/items/:lineId { "quantity": 3 }   -> 200 { cart } | 409 | 500
PATCH  /cart/items/:lineId { "wellnessPricingId" } -> 200 { cart }
DELETE /cart/items/:lineId                      -> 204
POST   /cart/merge { "items": guestCart }      -> 200 { cart, changes: [...] }   (you design `changes`)
POST   /checkout { "groupType", "items", "paymentMethod": "card" | "pay_at_clinic",
                   "idempotencyKey" }          -> 201 { orderId, status } | 402 | 409 | 500
GET    /orders/:orderId                         -> { orderId, status: "pending" | "paid" | "failed" }
```

The mock must support artificial latency and failure injection, for example through a small dev panel or query flags. You, and we, must be able to make a specific call fail or slow down on demand. `POST /checkout` must honour the idempotency key: the same key returns the same order rather than creating a new one.

Requirements

1. **Cart grouped by type,** with each group showing its own subtotal or subtotals. A group may contain several currencies: show each one separately, and never show a grand total that adds different currencies together. Design a summary that still helps the customer understand what they will pay.
2. **Quantity changes are optimistic.** The UI updates immediately. If the server rejects the change, the UI rolls back and tells the customer why. A customer who taps "+" five times quickly must end up with the right quantity, with no flicker, and the five requests must not race and overwrite each other out of order.
3. **Wellness items need a choice.** The customer must pick a duration option before that group can be checked out. Until then, the item's price is unknown, and it must not be counted as zero anywhere.
4. **Unavailable items** (an empty `product`) are clearly marked, can't be checked out, and are easy to remove.
5. **Split checkout.** The customer can check out a single group or everything at once. Each group becomes a separate order and may use a different payment method: card or pay at clinic.
6. **Partial failure is the core of this task.** If the customer checks out three groups and the second one fails:
7. the first must not be lost, or charged twice;
8. the customer must understand exactly what succeeded and what didn't;
9. they can retry only what failed;
10. a retry must never create a duplicate order. Use the idempotency key.
11. **Reload safety.** If the customer refreshes or closes the tab in the middle of checkout, coming back to the page must show the true state of each order. It must not restart from scratch.
12. **Guest cart.** When a guest logs in, their `guestCart` is merged into `serverCart`. The guest cart overlaps the server cart in several ways; look at the data. Design what happens in each case, and tell the customer what changed.

13. **States and accessibility.**
14. Loading, empty cart, slow network and offline states.
15. Every action is keyboard-accessible.
16. Focus is managed sensibly when an item is removed or a checkout step changes.

Hints on what we will look at closely

- Your state model for checkout. We suggest drawing it as a state machine in your README.
- How you prevent double charges.
- How you explain several orders and payments to a non-technical customer.

Questions?

If something is genuinely ambiguous, make a reasonable decision, write it down in your README and carry on. That is exactly what we'd want on the job. If you're blocked, email us at **[ADD HIRING EMAIL]** and we'll reply within one working day.

Good luck. We're looking forward to seeing how you think.